

WEEK 19: GRADED MINI PROJECT

Fashion-MNIST Generative Modelling

Denoising Autoencoder • Variational Autoencoder
Conditional GAN • StyleGAN-lite (AdaIN)

**Submitted by
Vivek Bhushan**

Objective

To build and compare four generative components on Fashion-MNIST and understand compression, probabilistic sampling, conditional control, and style mixing.

Project Overview

In this project, I used the Fashion-MNIST dataset to study four related ideas in generative modelling. I started with a denoising autoencoder, moved to a VAE for sampling and interpolation, trained a conditional GAN for class-controlled generation, and finally used AdaIN to demonstrate style mixing. I kept the architectures deliberately small because the images are only 28×28 pixels and the aim was to understand the behaviour of each method rather than chase a benchmark score.

The full experiment was run in Google Colab on a CUDA-enabled T4 GPU. I used the same data pipeline and visualisation helper across the tasks, which made it easier to compare outputs. The main result was not that one model “won,” but that each model solved a different part of the problem: the AE reconstructed reliably, the VAE gave a usable continuous latent space, the cGAN gave label-level control, and AdaIN showed how appearance statistics can be transferred.

Component	Main purpose	My main observation
Denoising AE	Recover a clean image from a noisy input	The noise was removed well, but fine details became smoother.
VAE	Learn a probabilistic latent space	Interpolation was smooth, while random samples were noticeably blurry.
cGAN	Generate an image for a requested class	Class shapes improved over training, although quality still varied by category.
AdaIN demo	Mix content structure with style statistics	Structure mostly remained, while contrast and texture changed.

Task 1: Setup

1.1 Environment

I enabled the GPU runtime before starting the experiment. The notebook selected CUDA successfully, so all four model components were trained or evaluated on the GPU. This was especially useful for the cGAN because each batch requires separate discriminator and generator updates.

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device) # Output: cuda
```

1.2 Libraries

I used PyTorch for the models and optimisation, torchvision for Fashion-MNIST, Matplotlib for the image grids, and tqdm for monitoring the epochs. I did not need an additional high-level training framework, which kept the training loops easy to inspect.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
```

1.3 Loading Fashion-MNIST

The train and test sets were downloaded through torchvision. I converted each grayscale image to a tensor, so the input shape became [1, 28, 28] and the pixel values remained between 0 and 1. The training loader was shuffled; the test loader was not. I used a batch size of 128 as suggested in the brief.

```
transform = transforms.Compose([transforms.ToTensor()])
train_ds = datasets.FashionMNIST('./data', train=True, download=True, transform=transform)
test_ds = datasets.FashionMNIST('./data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=128, shuffle=False)
```

1.4 Visualisation Helper

I wrote one helper that accepts either [B, 1, 28, 28] or [B, 28, 28] tensors. It moves the images back to the CPU, removes the one-channel dimension, and displays them without axes. I reused this helper for the AE, VAE, cGAN, and AdaIN results.

```
def show_grid(images, nrows=6, ncols=6, title=None):
    imgs = images.detach().cpu()
    if imgs.ndim == 4:
        imgs = imgs.squeeze(1)
    fig, axes = plt.subplots(nrows, ncols, figsize=(ncols, nrows))
    for i, ax in enumerate(axes.flat):
        if i < imgs.size(0):
            ax.imshow(imgs[i], cmap='gray')
            ax.axis('off')
    if title: fig.suptitle(title)
    plt.tight_layout(); plt.show()
```

Task 2: Denoising Autoencoder

2.1 Model Design

For the encoder, I used two stride-2 convolution layers. They reduce a 28×28 image to a $64 \times 7 \times 7$ feature map, which is flattened and projected to a 32-dimensional latent vector. The decoder reverses this process with a linear layer and two transposed convolutions. A final Sigmoid keeps reconstructed pixels in the same $[0, 1]$ range as the input.

```
self.enc = nn.Sequential(
    nn.Conv2d(1, 32, 3, stride=2, padding=1), nn.ReLU(),
    nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU())
self.enc_fc = nn.Linear(64*7*7, 32)
self.dec_fc = nn.Linear(32, 64*7*7)
self.dec = nn.Sequential(
    nn.ConvTranspose2d(64, 32, 3, 2, 1, output_padding=1), nn.ReLU(),
    nn.ConvTranspose2d(32, 1, 3, 2, 1, output_padding=1), nn.Sigmoid())
```

2.2 Adding Noise

I added fresh Gaussian noise in every training batch rather than creating one fixed noisy dataset. The noise level was $\sigma = 0.3$. After adding noise, I clamped the result so that values below 0 became 0 and values above 1 became 1.

```
def add_noise(x, sigma=0.3):
    noise = torch.randn_like(x) * sigma
    return torch.clamp(x + noise, 0.0, 1.0)
```

2.3 Training

The target was always the original clean image. I trained for 10 epochs using MSE loss and Adam with a learning rate of 0.001. The average loss dropped from 0.0309 in the first epoch to 0.0112 in the tenth. Most of the improvement happened early, followed by smaller but consistent gains.

Epoch	MSE loss
1	0.0309
2	0.0157
3	0.0138
4	0.0130
5	0.0125
6	0.0121
7	0.0118
8	0.0116
9	0.0114
10	0.0112

```
x_noisy = add_noise(x)
x_hat = ae(x_noisy)
loss = criterion_ae(x_hat, x)
opt_ae.zero_grad(); loss.backward(); opt_ae.step()
```

2.4 Outputs and Observations



Figure 1a. Clean Fashion-MNIST test images.

AE - Noisy (Figure 1b)



Figure 1b. The same test images after Gaussian noise was added.

AE - Denoised (Figure 1c)

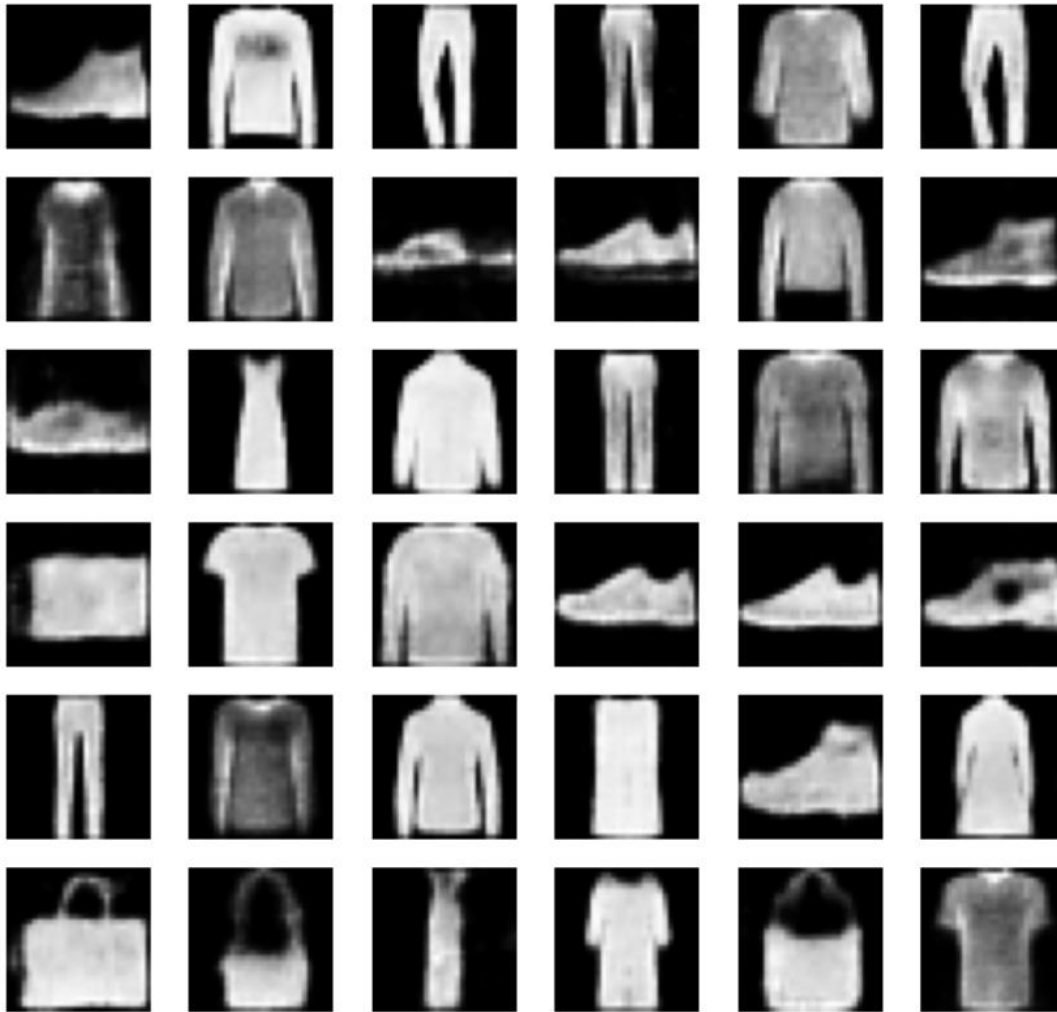


Figure 1c. Outputs reconstructed by the denoising autoencoder.

What I noticed:

- The denoised outputs recover the correct broad silhouette in nearly every example, including trousers, tops, shoes, and bags.
- Most of the isolated white noise disappears, and the black background becomes much cleaner.
- Edges are softer than in the original images. Logos, fabric patterns, straps, and shoe details are not reconstructed exactly.
- The model sometimes replaces unusual details with a more average-looking version of the class. This is clearest for patterned shirts and bags.
- The remaining blur is a reasonable trade-off here because the model is trained with pixel-wise MSE, which rewards an averaged reconstruction.

Task 3: Variational Autoencoder

3.1 Encoder Outputs

The VAE uses the same basic convolutional encoder shape as the autoencoder, but it has two separate linear heads. One predicts the 32-dimensional mean μ and the other predicts log variance. This allows one input image to map to a distribution rather than a single fixed point.

```
self.enc_fc_mu = nn.Linear(64*7*7, latent_dim)
self.enc_fc_logvar = nn.Linear(64*7*7, latent_dim)
mu = self.enc_fc_mu(h)
logvar = self.enc_fc_logvar(h)
```

3.2 Reparameterisation

Direct sampling would block gradient flow, so I used the reparameterisation trick. I sampled ϵ from a standard normal distribution and calculated $z = \mu + \sigma \epsilon$, where σ is obtained from log variance. In code, this keeps the random part separate from the learned parameters.

```
std = torch.exp(0.5 * logvar)
eps = torch.randn_like(std)
z = mu + std * eps
```

3.3 Training

The loss was the reconstruction BCE plus the KL regulariser. I trained for 12 epochs with Adam at 0.002. The average loss fell from 277.1204 to 242.0106. The fall was steep in the first two epochs and then became gradual, which suggests the model reached a fairly stable compromise between reconstruction and latent regularisation.

Epoch	ELBO loss
1	277.1204
2	251.7084
3	248.1188
4	246.3879
5	245.1764
6	244.4325
7	243.8168
8	243.3104
9	242.8749
10	242.5793
11	242.2650
12	242.0106

```
bce = F.binary_cross_entropy(x_hat, x, reduction='sum') / x.size(0)
kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0)
loss = bce + kl
```

3.4 Reconstructions and Samples



Original images used for reconstruction.

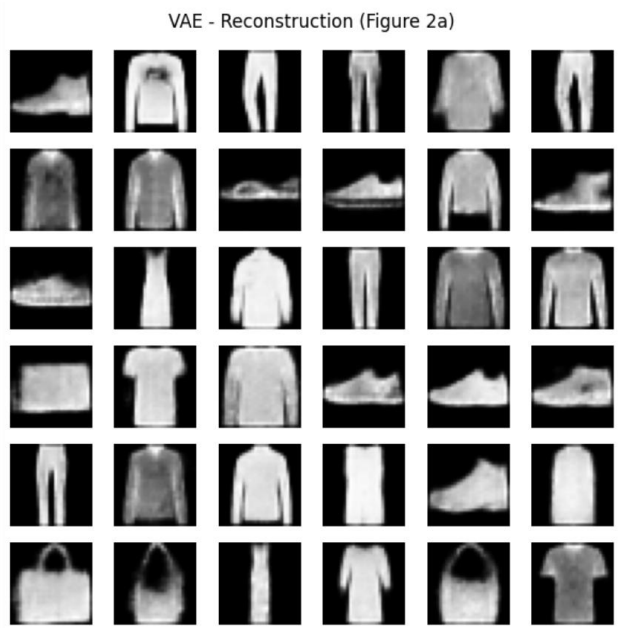


Figure 2a. VAE reconstructions.

VAE - Random Samples (Figure 2b)

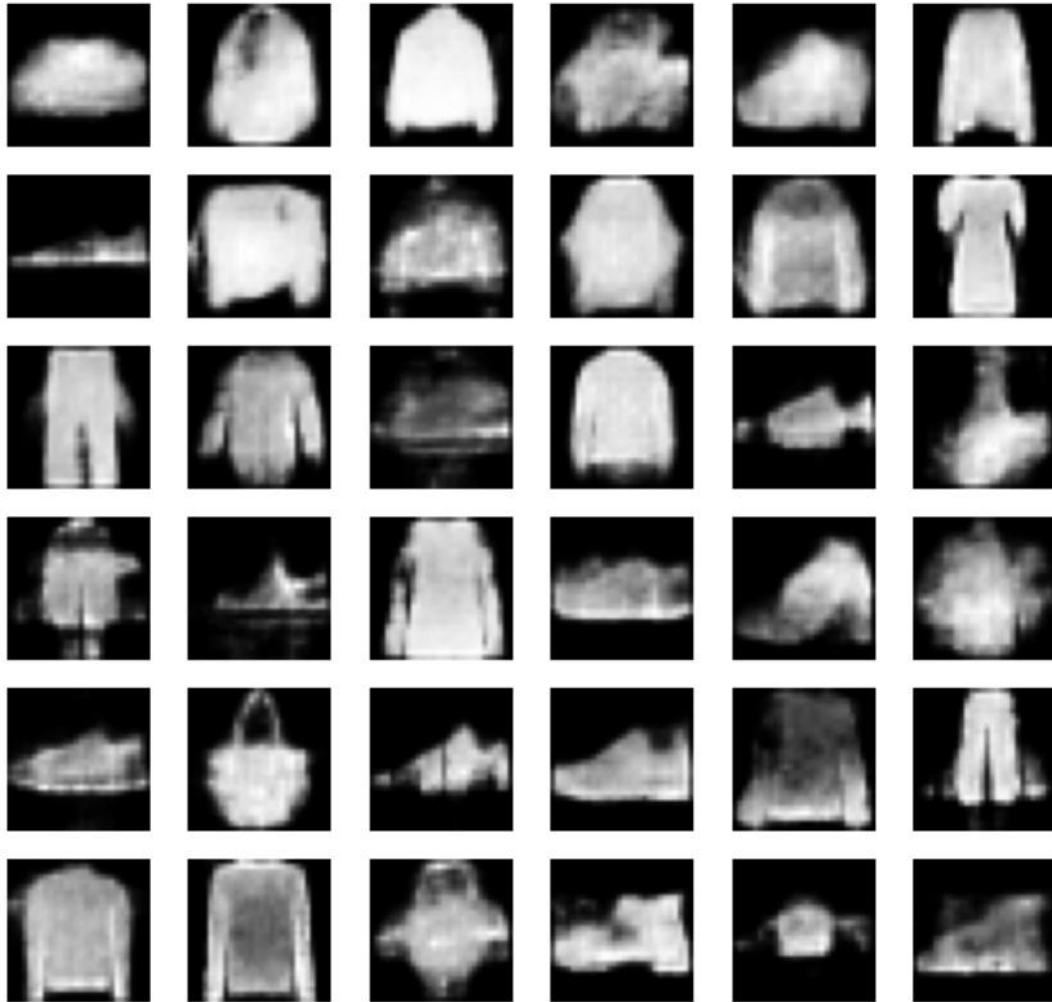


Figure 2b. Random samples decoded from $z \sim N(0, I)$.

The reconstructions preserve class and overall pose but are visibly softer than the clean originals. Random samples include recognisable shoes, tops, trousers, dresses, and bags, although some outputs sit between categories. The images are diverse enough to show that the decoder is not memorising a single prototype.

VAE - Interpolations (Figure 3)

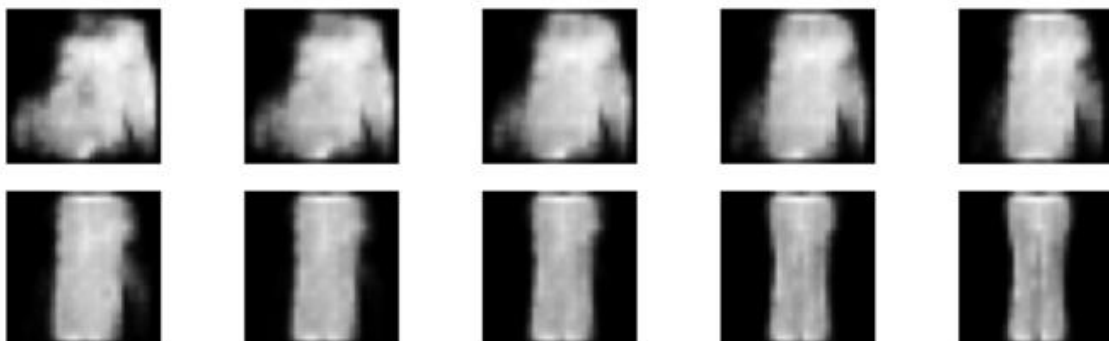
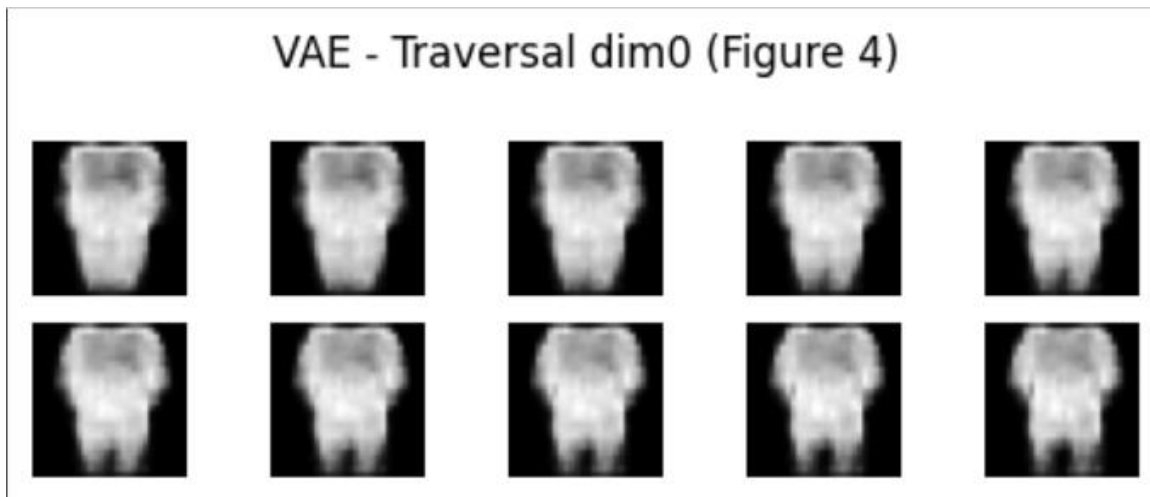


Figure 3. Ten steps of linear interpolation between two sampled latent vectors.

The interpolation changes smoothly from one endpoint to the other without sudden jumps. This is useful evidence that nearby points in the learned latent space decode to related images. The middle steps are slightly ambiguous, which is expected because they combine features from both endpoints.

**Figure 4. Traversal of latent dimension 0 from -3 to $+3$.**

Changing only dimension 0 produces a gradual visual change rather than a class switch. In this run, the effect is mostly on width, brightness, and local shape. It does not correspond to a clean human concept, so the latent representation is structured but not fully disentangled.

AE and VAE: What Changed?

- The AE gives the sharper reconstruction because every input maps to one deterministic latent vector.
- The VAE adds uncertainty and regularisation, so its reconstructions are smoother but its latent space can be sampled.
- The AE is better for the denoising objective used here; the VAE is more useful when new images and latent-space operations are required.
- Both models favour broad garment shape over small texture details.
- The VAE's interpolation is much more meaningful because the latent space is explicitly encouraged to follow a standard normal prior.

Task 4: Conditional GAN

4.1 Architecture

The generator receives a 64-dimensional random vector and a learned 10-dimensional embedding of the requested class label. I concatenate them, project the result to $256 \times 7 \times 7$, and upsample to 28×28 . The discriminator also receives the label: its embedding is expanded to a 28×28 map and concatenated with the image as a second channel. This means both networks are explicitly told which class is being judged.

```
# Generator conditioning
y = self.embed(labels)
x = torch.cat([z, y], dim=1)

# Discriminator conditioning
y_map = self.label_fc(self.embed(labels)).view(-1, 1, 28, 28)
x = torch.cat([img, y_map], dim=1)
```

4.2 Training

I trained the generator and discriminator alternately for 12 epochs using BCEWithLogitsLoss. Both optimisers used Adam with learning rate 0.0002 and betas (0.5, 0.999). I used 0.9 as the real target instead of 1.0, which is a small label-smoothing step. The losses fluctuate rather than decrease monotonically because each network is adapting to the other.

Epoch	D loss	G loss
1	1.2137	1.3112
2	1.2206	0.9020
3	1.2481	0.9780
4	1.3488	0.8429
5	1.3500	0.8222
6	1.3223	0.8665
7	1.3612	1.0070
8	1.3743	0.8491
9	1.3753	0.9006
10	1.3405	0.8679
11	1.3381	0.9196
12	1.3942	0.8847

```
# Discriminator update
real_targets = torch.full_like(real_logits, 0.9)
fake_targets = torch.zeros_like(fake_logits)
loss_D = criterion_gan(real_logits, real_targets) + criterion_gan(fake_logits, fake_targets)

# Generator update
gen_targets = torch.ones_like(gen_logits)
loss_G = criterion_gan(gen_logits, gen_targets)
```

4.3 Class-Controlled Samples

I generated one fresh image for each label after every epoch. The early grid is quite noisy, but the class layout is already starting to appear. By the middle of training, shoes and long garments are easier to identify. The final epoch is cleaner, although a few classes still have rough edges and uneven texture.

cGAN - Class Samples (Figure 5, epoch 1)

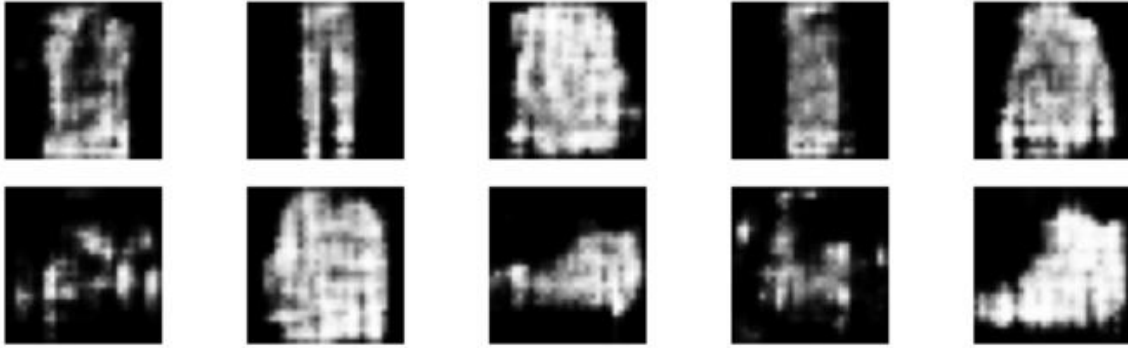


Figure 5a. Class samples after epoch 1.

cGAN - Class Samples (Figure 5, epoch 6)

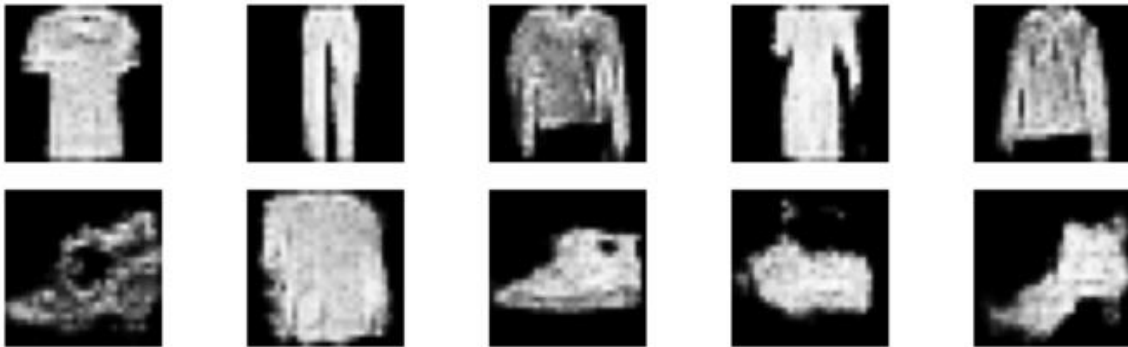


Figure 5b. Class samples after epoch 6.

cGAN - Class Samples (Figure 5, epoch 12)



Figure 5c. Class samples after epoch 12.

cGAN - Diversity Grid (Figure 6)



Figure 6. Four independently generated examples for each class label.

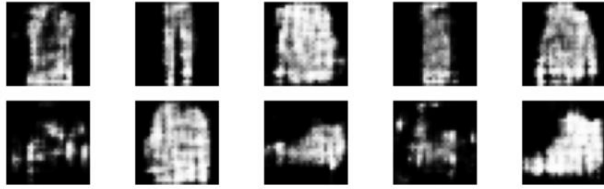
Conditioning observations:

- Changing the label changes the overall category even when the random noise is newly sampled.
- Trousers, dresses, and ankle boots are easier to distinguish because their silhouettes are very different.
- T-shirts, shirts, pullovers, and coats remain more easily confused because their 28×28 outlines overlap.
- The diversity grid contains several variations inside most labels, so the generator has not collapsed to exactly one output per class.
- Some classes still show repeated brightness patterns, suggesting partial mode preference rather than complete mode collapse.
- The generated images are often sharper than VAE samples, but they also contain more irregular local artifacts.

cGAN Training Progression

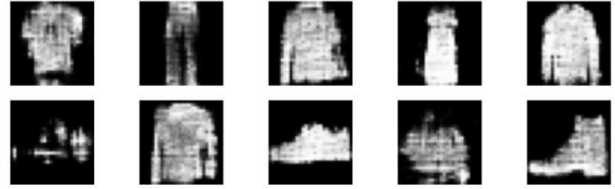
The following grids record every epoch that was saved. I included them because they show the uneven nature of adversarial training more honestly than selecting only the best-looking epoch. Image quality does not improve in a perfectly straight line, but the class silhouettes become more dependable overall.

cGAN - Class Samples (Figure 5, epoch 1)



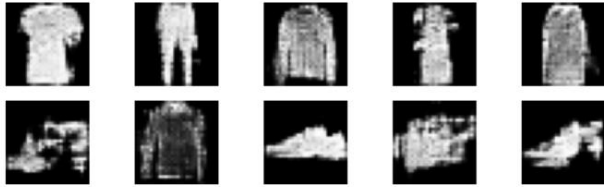
Epoch 1

cGAN - Class Samples (Figure 5, epoch 2)



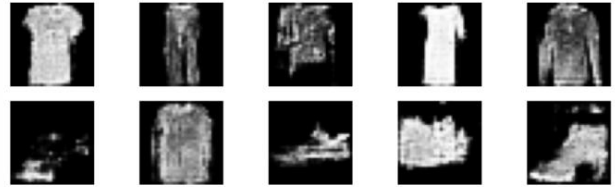
Epoch 2

cGAN - Class Samples (Figure 5, epoch 3)



Epoch 3

cGAN - Class Samples (Figure 5, epoch 4)



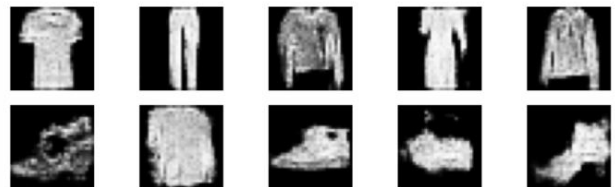
Epoch 4

cGAN - Class Samples (Figure 5, epoch 5)



Epoch 5

cGAN - Class Samples (Figure 5, epoch 6)



Epoch 6

cGAN - Class Samples (Figure 5, epoch 7)



Epoch 7

cGAN - Class Samples (Figure 5, epoch 8)



Epoch 8

cGAN - Class Samples (Figure 5, epoch 9)



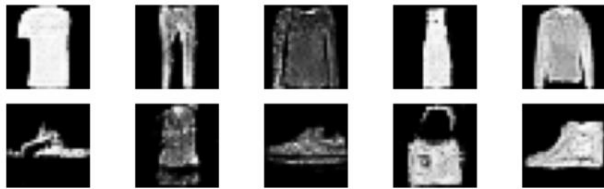
Epoch 9

cGAN - Class Samples (Figure 5, epoch 10)



Epoch 10

cGAN - Class Samples (Figure 5, epoch 11)



Epoch 11

cGAN - Class Samples (Figure 5, epoch 12)



Epoch 12

Task 5: StyleGAN-lite AdaIN Demo

5.1 Implementation

This section is a lightweight demonstration of the style-mixing idea rather than a complete StyleGAN. I passed eight content images and eight style images through a small two-layer feature extractor. AdaIN then normalised the content features and rescaled them with the mean and standard deviation of the corresponding style features. A small convolutional decoder converted the mixed features back to image space.

```
def adain(content, style, eps=1e-5):  
    c_mean = content.mean((2, 3), keepdim=True)  
    c_std = content.std((2, 3), keepdim=True) + eps  
    s_mean = style.mean((2, 3), keepdim=True)  
    s_std = style.std((2, 3), keepdim=True) + eps  
    normalized = (content - c_mean) / c_std  
    return normalized * s_std + s_mean
```

5.2 Content, Style, and Mixed Outputs

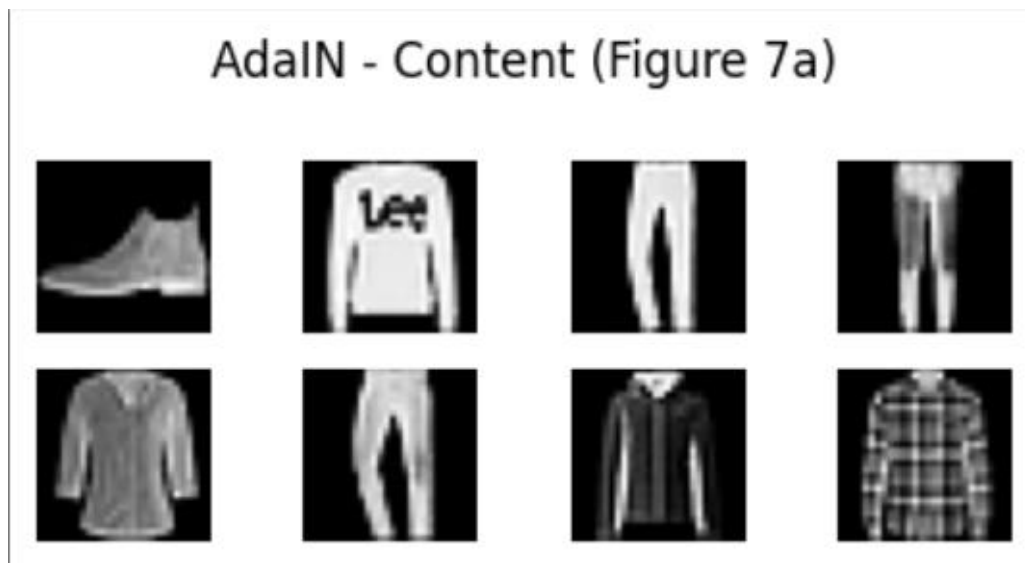


Figure 7a. Content images that supply the main spatial structure.

AdaIN - Style (Figure 7b)



Figure 7b. Style images that supply feature statistics.

AdaIN - Mixed (Figure 7c)

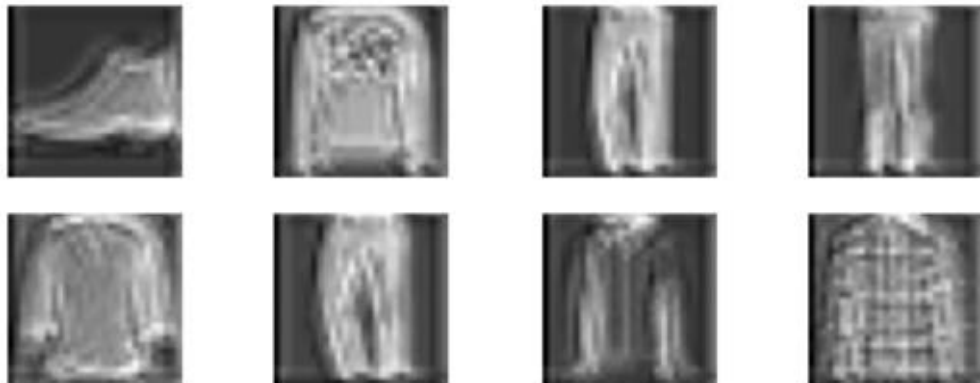


Figure 7c. Images decoded after AdaIN mixing.

The mixed results generally keep the content item in the same location and retain its broad outline. What changes most is contrast, brightness, edge emphasis, and texture. Some pairs work better than others: when the content and style classes are very different, the mixed output becomes less clear. Because Fashion-MNIST is grayscale and low resolution, there is no colour palette to transfer, so the style effect is much more limited than it would be on natural images.

Analysis and Short Report

AE vs VAE

The main difference is the latent representation. The AE maps an input directly to one 32-dimensional vector and decodes that same vector, so it is deterministic. This worked well for denoising because the objective was simply to recover the original image. The VAE maps an input to a distribution described by μ and log variance, then samples from it. That extra constraint makes the latent space smoother and sampleable, but it also reduces reconstruction sharpness.

VAE vs cGAN

The VAE is trained with an explicit reconstruction-plus-KL objective. It learns a regular latent space and provides stable interpolation, but its generated images are averaged and soft. The cGAN does not directly minimise pixel reconstruction error; instead, the generator learns through the discriminator's feedback. This produced stronger class silhouettes in several cases, but the training losses moved up and down and the image quality was less predictable.

Why Labels Improve Control

Without labels, random noise only tells the generator to make some plausible Fashion-MNIST image. In the cGAN, the label embedding gives the generator a clear category target. The discriminator receives the same label and can reject an image that looks realistic but does not match the requested class. This two-sided conditioning is why a single trained model can generate examples for all ten categories.

What Each Model Captured

Model	Captured well	Captured poorly
Denoising AE	Silhouette, background cleanup, common shapes	Logos, thin straps, small texture and sharp edges
VAE	Smooth latent transitions and broad garment identity	Sharp detail and fully distinct random samples
cGAN	Class-controlled silhouettes and within-class variation	Stable texture; separation of similar upper-body classes
AdaIN demo	Brightness, contrast, and simple texture statistics	Rich semantic style and colour information

Practical Notes

- The noise standard deviation of 0.3 was strong enough to make denoising visible without completely hiding most objects.
- A batch size of 128 gave 469 training batches per epoch and worked comfortably on the T4 GPU.
- The AE learning rate of 0.001 converged quickly and then improved gradually without instability.
- The VAE learning rate of 0.002 worked, but reconstruction remained soft because of the probabilistic objective, not simply because of insufficient training.
- The lower cGAN learning rate of 0.0002, Adam betas (0.5, 0.999), batch normalisation, and real-label smoothing helped keep adversarial training usable.

- For the cGAN, generated grids were more informative than reading either loss alone. A higher or lower GAN loss did not always mean better-looking samples.

Conclusion

This project made the differences between reconstruction models and generative models much clearer to me. The denoising AE was the most straightforward to train and gave the cleanest reconstructions. The VAE sacrificed some sharpness but gave a latent space that could be sampled, interpolated, and traversed. The cGAN gave the best direct control over the output class, although it needed more careful training and still produced artifacts. The AdaIN experiment was simple, but it clearly showed that feature statistics can change appearance while retaining much of the original content structure.

If I extended the work, I would train the cGAN for more epochs, save fixed-noise samples for a fairer epoch-to-epoch comparison, and compute quantitative measures alongside the image grids. For the VAE, I would test the effect of changing the latent dimension and KL weight. For AdaIN, using a trained encoder-decoder and higher-resolution colour images would make the style transfer result much easier to judge.

Submission Checklist

Requirement	Included
Environment, imports, data loading, and visualisation helper	Yes
Denoising AE model, noise, training, Figure 1, observations	Yes
VAE distribution, reparameterisation, ELBO, Figures 2–4	Yes
cGAN architecture, training, class samples, diversity grid	Yes
AdaIN implementation and Figure 7	Yes
Model comparisons, takeaways, and practical notes	Yes

The complete runnable implementation and raw training outputs are retained in the submitted Jupyter notebook. This report focuses on the experiment design, observed behaviour, and the figures requested in the project brief.